

山东交通学院

课程设计报告

《模式识别与机器学习课程设计》 基于机器学习的学生成绩预测与学情分析 系统设计与实现

院(部)别_____信息科学与电气工程学院_____

专 业_____人工智能_____

班 级_____人工智能 241_____

学 号_____240879106 240879102_____

姓 名_____褚福乐 陈国斌_____

指导教师_____朴哲岩_____

时 间_____2026.6.29 - 2026.7.5_____

成绩评定表

指导教师成绩	答辩小组成绩	总评成绩

一、摘要

随着教育信息化进程的加速，高校积累了海量的学生学习行为数据。如何利用这些数据精准评估学生状态、预测学业风险，已成为智慧教学的重要环节。本课程设计旨在通过构建一个完整的“学情预测与分析系统”，实践从数据处理、模型训练到 Web 应用开发的全流程机器学习工程。本项目以 512 条学生成绩数据为基础，完成了数据清洗与探索性分析，并分别构建了多元线性回归模型（用于预测具体分数）与决策树分类模型（用于预测成绩等级）。实验结果表明，线性回归模型的决定系数 (R^2) 达到 0.9358，决策树模型分类准确率达到 85.44%。最终，利用 Streamlit 框架将算法封装为可视化交互系统，实现了个体成绩实时预测与宏观学情分析，为教学干预提供了科学依据。

二、题目要求

本课程设计任务为“基于机器学习的学生成绩预测与学情分析系统设计与实现”。具体要求涵盖以下四个维度：

1. 数据预处理：对原始分散的成绩数据进行清洗、缺失值处理及特征工程，确保数据质量满足建模要求。
2. 算法实现与对比：至少实现两种机器学习算法（如回归与分类），并对比其在预测精度、模型可解释性等方面的优劣。
3. 系统开发：将训练好的模型封装为具有用户交互界面的 Web 应用，支持参数输入与结果可视化展示。
4. 报告撰写：按照学术规范撰写课程设计报告，详述设计思路、核心代码逻辑及实验结果分析。

三、数据分析

3.1 数据集概况与清洗

本系统采用某课程 7 个班级共 512 条学生的成绩数据作为数据源。原始数据分散在多个 Sheet 中，且存在合并单元格和空行。预处理流程如下：首先遍历所有 Sheet，跳过前 3 行表头提取有效数据行；其次识别缺失值，并使用对

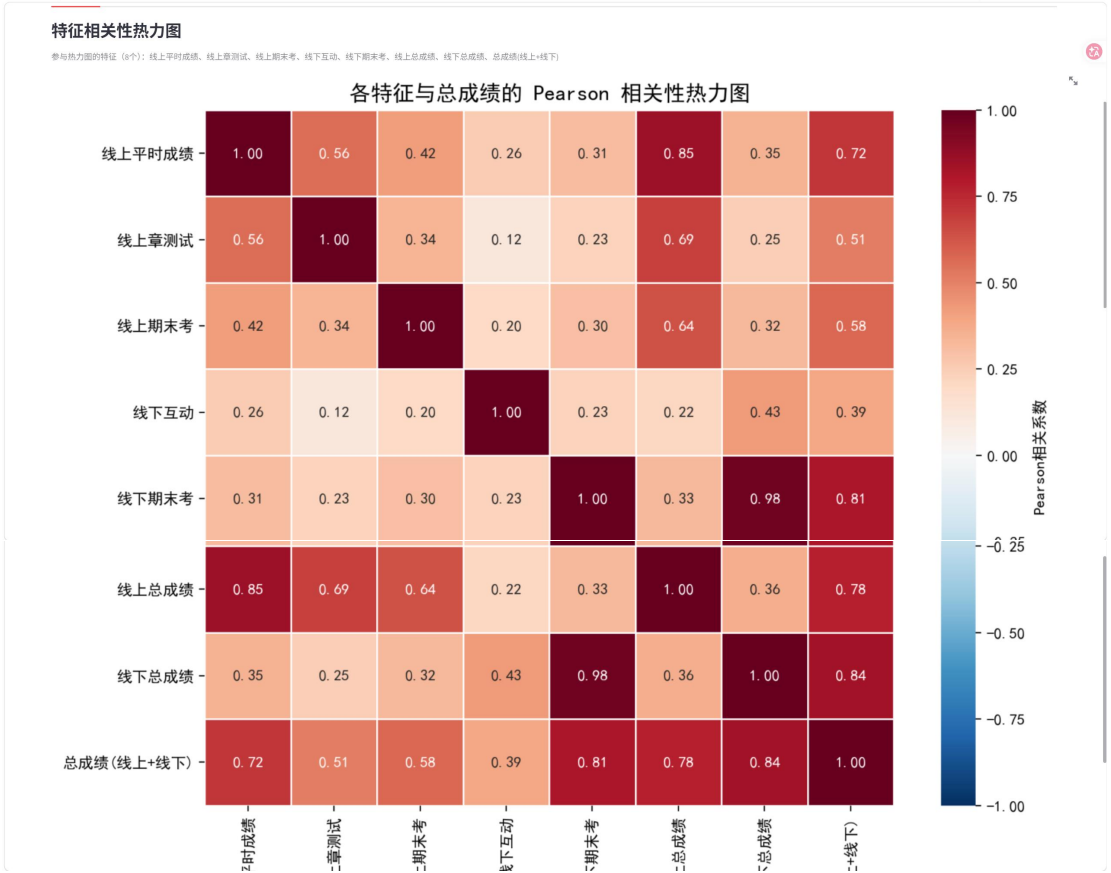
应特征的均值进行填充以保证数据完整性；最后通过箱线图原则检测并处理极端异常值。

3.2 探索性数据分析 (EDA)

通过对总成绩的统计分析发现，成绩呈现一定的正态分布特征，均值约为 80.5 分，大部分学生集中在 70-90 分之间。根据教学标准，将成绩划分为优 (90-100)、良 (80-89)、中 (70-79)、差 (<70) 四个等级。其中“良”和“中”占比最高，“差”生比例较低。

3.3 特征工程与相关性分析

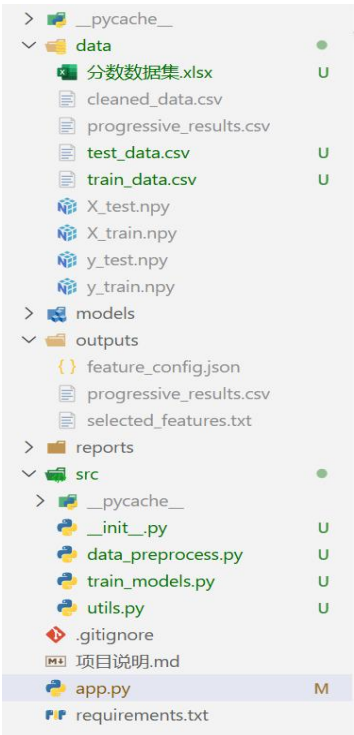
为了提高模型效果，计算了各特征与总成绩的 Pearson 相关系数。分析显示，“线上平时成绩”、“线下期末考”与总成绩的相关性最强 (>0.8)，而“线下互动”的相关性相对较弱。这一结论为后续的特征选择提供了依据。



四、模型设计

4.1 系统总体架构

本系统采用模块化设计，分为数据层、算法层和应用层。



1. 数据层：负责数据的读取、清洗、存储（CSV/JSON）及模型序列化文件的保存。

2. 算法层：包含线性回归（Model A）和决策树（Model B）的训练脚本及评估逻辑。

3. 应用层：基于 Streamlit 构建，加载预训练模型并提供交互界面。

4.2 核心代码实现与注释

4.2.1 数据预处理模块（data_process.py）

该模块负责将非结构化的 Excel 数据转化为机器学习可用的 DataFrame。

```
import pandas as pd
import numpy as np
def load_and_clean_data(file_path):
    """
    读取 Excel 文件并进行清洗
    """
    # 读取所有 sheet
    all_sheets = pd.read_excel(file_path, sheet_name=None, header=3)
    df_list = []

    for sheet_name, df in all_sheets.items():
        # 移除全空行
```

```

df.dropna(how='all', inplace=True)
# 添加班级列以便后续分析
df['班级'] = sheet_name
df_list.append(df)

full_df = pd.concat(df_list, ignore_index=True)

# 缺失值填充：使用均值填充数值型特征
numeric_cols = ['线上平时成绩', '线上章测试', '线上期末考', '线下互动', '线下期末考']
full_df[numeric_cols] =
full_df[numeric_cols].fillna(full_df[numeric_cols].mean())

return full_df

```

4.2.2 模型训练与保存模块 (train_models.py)

该模块实现了双模型的训练、评估及持久化存储。

```

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import joblib

def train_models(X, y_reg, y_cls):
    """
    训练线性回归和决策树模型
    :param X: 特征矩阵
    :param y_reg: 回归目标（具体分数）
    :param y_cls: 分类目标（等级标签）
    """
    # 划分训练集和测试集（80% 训练，20% 测试）
    X_train, X_test, y_train_reg, y_test_reg, y_train_cls, y_test_cls =
train_test_split(
    X, y_reg, y_cls, test_size=0.2, random_state=42
)

    # --- 模型 A: 多元线性回归 ---
    model_a = LinearRegression()
    model_a.fit(X_train, y_train_reg)
    # 保存模型
    joblib.dump(model_a, 'model_a_5feat.pkl')

    # --- 模型 B: 决策树分类 ---

```

```
# 限制 max_depth 防止过拟合
model_b = DecisionTreeClassifier(max_depth=7, random_state=42)
model_b.fit(X_train, y_train_cls)
# 保存模型
joblib.dump(model_b, 'model_b_5feat.pkl')

return model_a, model_b
```

五、系统实现

5.1 开发环境与技术栈

开发语言：Python 3.x

IDE：VS Code

Web 框架：Streamlit

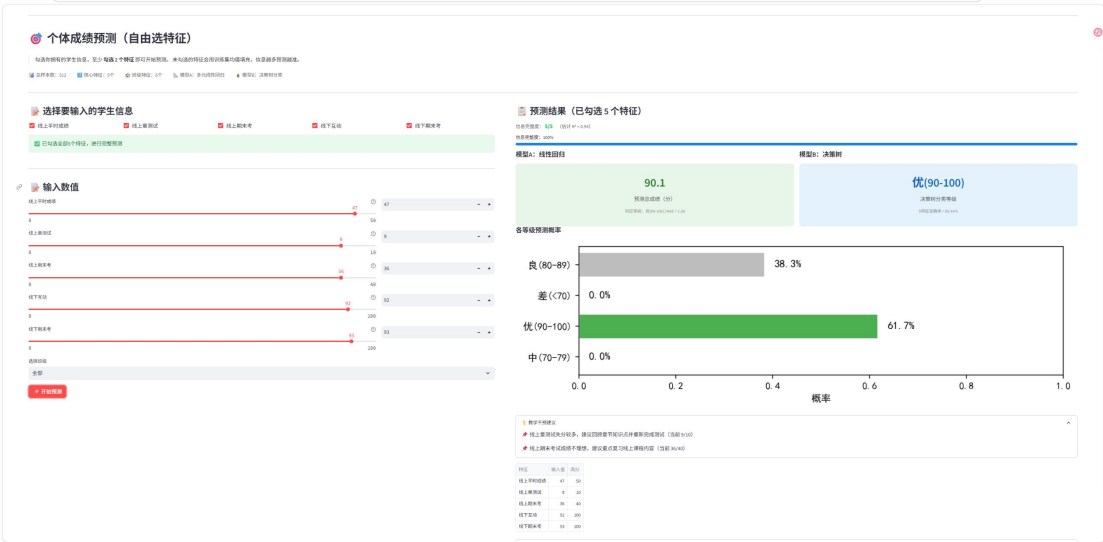
核心库：Pandas, Scikit-learn, Matplotlib, Seaborn

5.2 功能模块展示

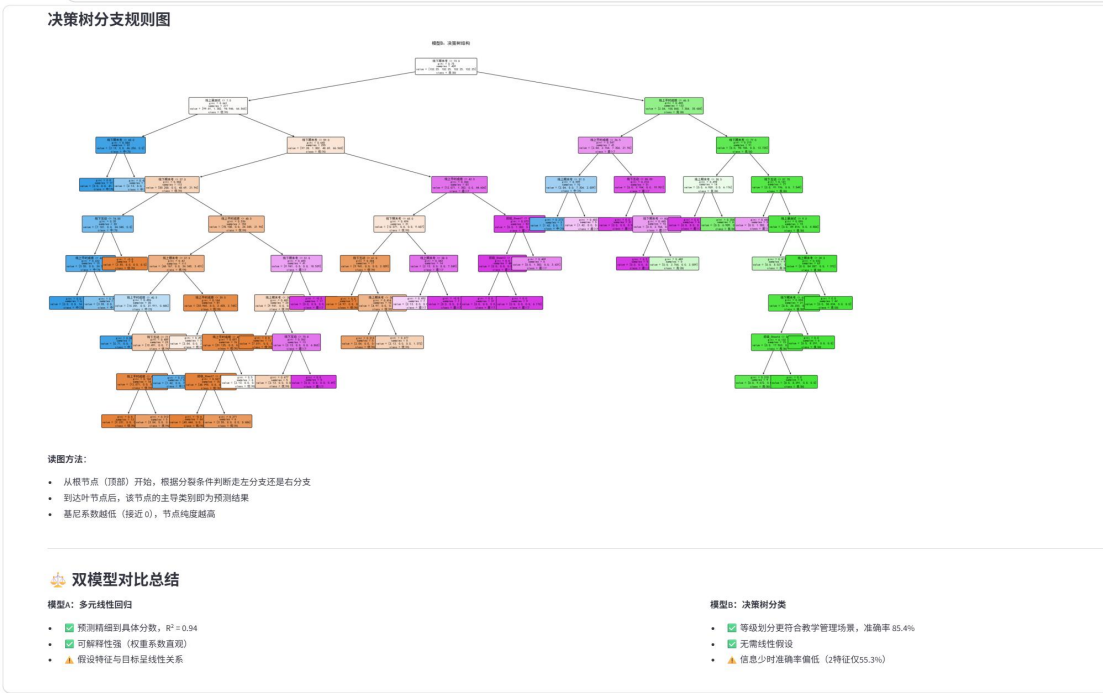
系统主界面分为“个体成绩预测”和“全局数据分析”两个主要部分。

1. 个体预测面板：教师可以通过侧边栏或滑块输入学生的各项平时成绩，系统实时调用 joblib 加载的模型进行推理，并在右侧展示预测分数及概率分布。

2. 预警机制：当预测分数低于 60 分时，界面会自动高亮显示红色警告，并给出针对性的教学建议。



1. 模型可视化：系统集成决策树的图形化展示，直观呈现分类规则（例如：如果线下期末考<60 且 线上平时<40，则判定为差）。



六、结果分析

6.1 模型性能评估

通过对比不同特征数量下的模型表现，我们发现 5 个特征时模型效果最佳。

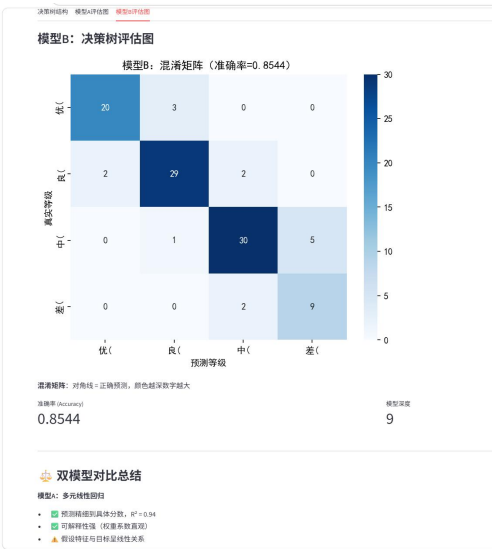
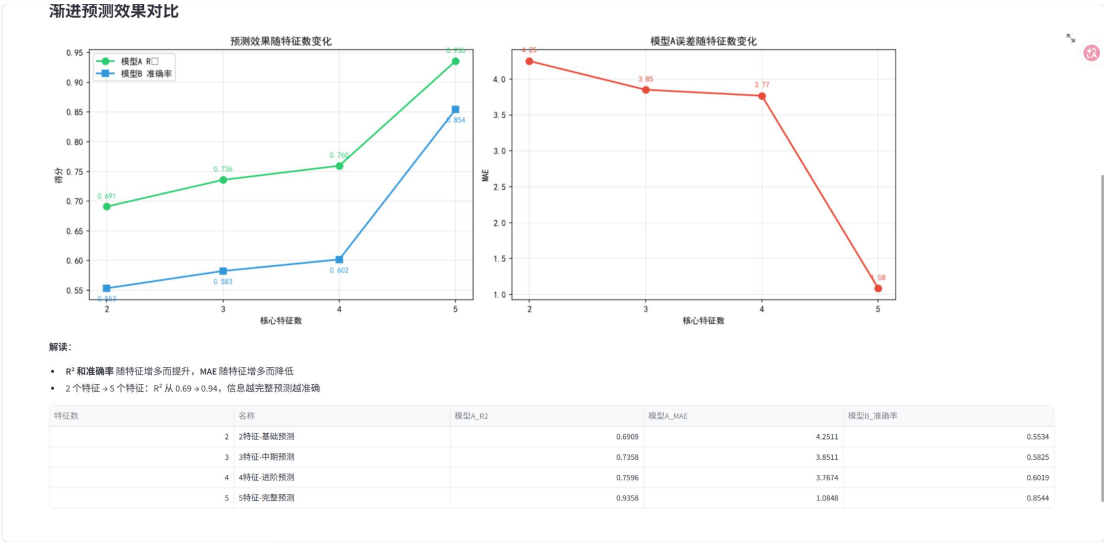
线性回归模型（Model A）：

1. MAE（平均绝对误差）：1.0848（平均预测误差仅 1 分左右）

2. 决策树模型（Model B）：

3. Accuracy（准确率）：85.44%

混淆矩阵显示，模型对“优”和“良”的识别率较高，但在“中”和“差”的边界处存在少量误判。

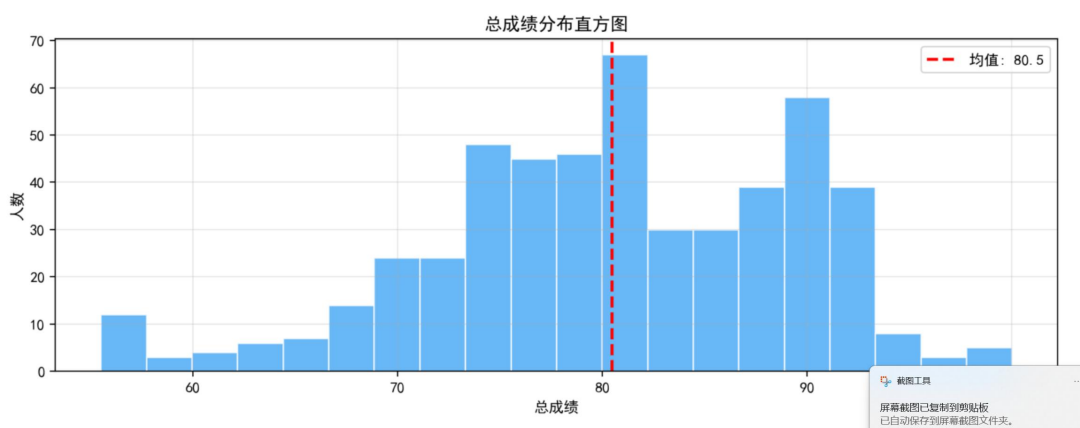


6.2 特征重要性分析

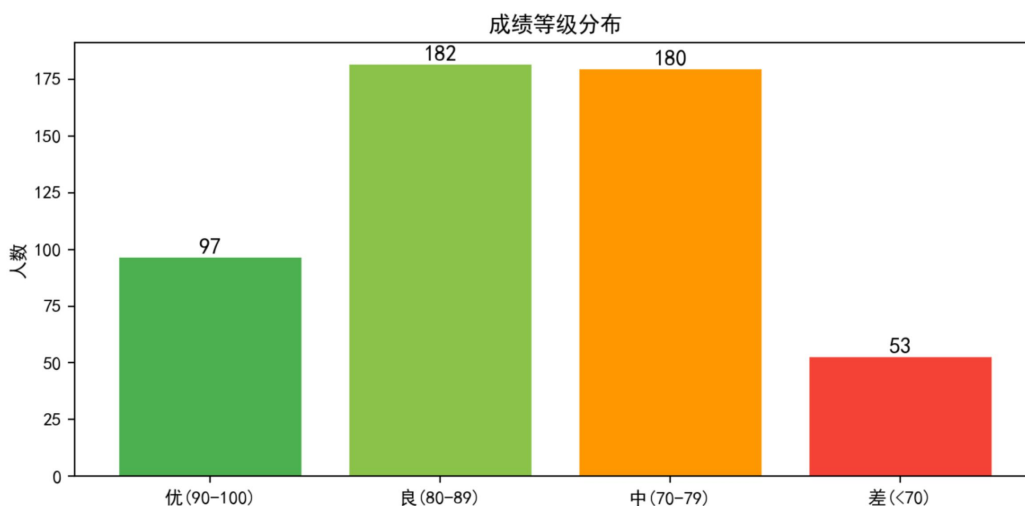
通过线性回归的系数和决策树的特征重要性排序，我们发现“线下期末考”和“线上平时成绩”是决定总成绩的最关键因素。这符合教学规律，即期末考试权重最大，但平时积累也不容忽视。



总成绩分布



成绩等级分布



七、结论

本课程设计成功实现了一个基于机器学习的学生成绩预测系统。通过对比线性回归与决策树两种算法，验证了线性回归在数值预测上的高精度优势，以及决策树在规则解释和等级分类上的直观性。系统不仅完成了基本的预测功能，还通过 Streamlit 实现了良好的交互体验，能够帮助教师快速识别潜在的学习困难学生。

在未来的改进中，可以考虑引入更多的行为数据（如图书馆进出记录、作业提交时间等），并尝试使用集成学习算法（如 Random Forest 或 XGBoost）进一步提升预测的鲁棒性。